



**Tribhuvan University
Institute of science and technology
AMRIT SCIENCE CAMPUS**

**Final Year Project Proposal Report
On
Automated Document Validation System**

Under the Supervision of
Asst. Prof. Dabbal Singh Mahara
Department of Computer Science & Information Technology
Amrit Campus, Thamel, Kathmandu

A Final Year Project Proposal Report submitted in partial fulfilment of the requirements for the degree of Bachelor in Information Technology awarded by Tribhuvan University.

Submitted by:

Ashlesha Thapa (79020112)
Sakshi Sapkota (79020132)
Samundra Prasad Neupane (79020133)

Submitted to:

Department of Computer Science & Information Technology
Amrit Campus, Thamel, Kathmandu
Tribhuvan University Lainchaur, Kathmandu
Falgun, 2082

TABLE OF CONTENTS

1. INTRODUCTION	1
2. PROBLEM STATEMENT	2
3. OBJECTIVES	2
4. LITERATURE REVIEW	3
5. METHODOLOGY	4
5.1 REQUIREMENT IDENTIFICATION	4
5.1.1 Study of existing system	4
5.1.2 Requirement Collection	4
5.2 FEASIBILITY STUDY	6
5.3 HIGH-LEVEL DESIGN OF SYSTEM	7
5.3.1 Methodology of the Proposed System	7
5.3.2 Workflow of the Proposed System:	7
5.3.3 System Flowchart	8
5.3.4 System Architecture	9
5.3.5 Description of Algorithms	9
6. EXPECTED OUTCOMES	10
7. REFERENCES	11

1. Introduction

Digital documents are widely used in schools, offices, and other organizations. Documents like reports, theses, resumes, and official records are important for communication, evaluation, and decision-making. These documents need to follow specific structure and formatting rules. Currently, checking documents is mostly done by hand. This manual validation takes a lot of time, can be inconsistent, and often has errors, especially when there are many documents to check. For example, a university department may handle over 2,000 student reports and theses each semester, with manual checking taking around 20 minutes per document. Studies show that human reviewers can miss up to 15–25% of formatting or structural errors in long documents [3].

Manual checking also cannot easily handle large numbers of documents. As more organizations accept digital submissions, reviewers face a growing workload while trying to stay accurate and fair. This makes manual validation hard to manage on a large scale. Automated document verification systems can help by saving time, providing consistent checks, and reducing errors. Research shows that automated systems can read documents and find errors faster and more reliably than humans [1], [4]. In fact, studies suggest that automation can cut document evaluation time by up to 70% while maintaining accuracy [2].

Current tools like grammar checkers, plagiarism detectors, and document editors only help with certain parts of a document. They do not check everything, such as overall structure, completeness, formatting, or custom rules. Automated document validation systems can analyze documents and identify missing sections, formatting mistakes, or structural problems [2]. Recent technologies, like layout-aware models and document parsing systems, make it possible to automate document analysis even better [5], [6].

This project proposes a **Document Requirement Validation System**. It will automatically check uploaded documents against predefined rules or user-provided requirement files. The system will find missing sections, formatting errors, and structural issues. It will provide a validation score, section-wise feedback, and suggestions to improve the document. The system can also make small corrections automatically, like adding missing headings or fixing spacing. By using this system, users can save time, get consistent checks, and produce high-quality, properly formatted documents.

2. Problem Statement

Documents such as reports, theses, resumes, and cover letters are widely used in academic and professional environments. Ensuring that these documents follow proper structure, formatting, and content requirements is essential for clarity, professionalism, and compliance with submission guidelines.

Currently, document validation is mostly performed manually by instructors, administrators, or HR personnel. This process is time-consuming, inconsistent, and prone to errors. Reviewers may overlook missing sections, formatting issues, or structural mistakes, and feedback is often limited or unclear.

Existing tools only address specific aspects of document quality. Grammar checkers focus on language, plagiarism detectors check originality, and word processors provide formatting tools, but none offer a complete evaluation of structure, formatting, completeness, and adherence to requirements. Users often need multiple tools while still performing manual checks, reducing efficiency and consistency.

There is therefore a need for an **automated document validation system** that can handle multiple file formats, verify structure and formatting, compare content against requirements, and provide clear, actionable feedback. Such a system would reduce manual effort; improve consistency, and help users produce high-quality, standards-compliant documents.

3. Objectives

The main objectives of the proposed Document Requirement Validation System are:

- i. To design and develop an automated system that validates documents by analyzing structure, formatting, and completeness against predefined or user-specified requirements.
- ii. To implement a multi-format processing framework capable of extracting and evaluating content from PDF and DOCX documents efficiently.
- iii. To generate structured validation reports containing overall scores, section-wise analysis, and actionable feedback to improve document quality.

4. Literature Review

Several approaches have been developed to address the challenges of document validation. Early solutions relied on rule-based methods, which applied fixed rules to check headings, spacing, sections, and formatting. These systems are effective for standard documents, and can also generate scores or reports to help users identify weak areas and improve their documents more efficiently than manual checking [2]. However, they struggle with variations in structure, document type, or user-specific requirements.

Other tools focus on specific aspects of document quality. Grammar checkers like Grammarly or LanguageTool ensure correct language, plagiarism detection systems check originality, and document editors assist with formatting. Yet, these tools do not provide a comprehensive evaluation, as they cannot fully check for completeness, adherence to rules, or section-wise structure [2], [3].

Advances in document analysis and natural language processing (NLP) have enabled systems to process text and structure together. NLP models can classify sections, detect missing content, and identify formatting inconsistencies [4]. Layout-aware models further improve automated evaluation by understanding text position, headings, tables, and other visual features [5], [6]. These methods make automated validation more accurate and capable of handling multiple document types, including reports, theses, resumes, and cover letters.

Despite these advancements, most existing tools still focus on only one part of a document, such as grammar, plagiarism, or basic formatting. Few systems provide a complete solution that checks structure, formatting, completeness, and compliance with custom requirements simultaneously. This gap highlights the need for a **comprehensive Document Requirement Validation System** that combines rule-based checks, multi-format support, section-wise scoring, and actionable feedback [2]. By addressing these limitations, automated validation systems can save time, improve document quality, and provide consistent evaluation across large numbers of files.

5. Methodology

5.1 Requirement Identification

5.1.1 Study of existing system

- **Manual Document Review:** Most academic and professional institutions rely on manual validation performed by instructors, reviewers, or administrative staff. While this approach allows for detailed assessment, it is time-consuming, inconsistent, and difficult to scale, especially when large numbers of documents must be checked within a short timeframe.
- **Single-Purpose Validation Tools:** Some software tools assist with specific validation tasks, such as grammar correction, plagiarism detection, or basic formatting checks. Although useful, these tools do not provide comprehensive evaluation of structural completeness, section-wise organization, or compliance with document requirements, leaving gaps that still require manual review.
- **Format-Dependent Systems:** Many existing systems are limited to specific document formats, such as DOCX or PDF, or support only certain types of documents. This restricts their applicability across different workflows, making them less suitable for institutions or organizations that deal with a variety of document types like reports, theses, resumes, and cover letters.
- **NLP-Based and Automated Validation Tools:** Some modern platforms use natural language processing to analyze text, detect missing sections, classify document components, or check compliance with templates. Examples include DocVerify [2] and other automated document verification services [3]. These tools reduce manual effort and improve consistency, but most are either limited in format support, costly, or cannot provide detailed section-wise feedback, leaving room for a more flexible and comprehensive solution.

5.1.2 Requirement Collection

a) Functional Requirements

- i. **Document Validation Engine:** The system accepts user-submitted documents in PDF or DOCX format. It extracts the text, identifies document sections, and checks for compliance with either predefined formatting rules or user-uploaded requirement specifications. Each section is analyzed for completeness, headings, spacing, alignment, and formatting consistency. The system generates section-wise scores and an overall validation score to indicate document quality.
- ii. **Feedback and Auto-Correction Module:** The system provides actionable feedback for each section that fails to meet the required standards. Basic auto-corrections, such as inserting missing headings, adjusting spacing, or aligning formatting, are applied automatically. Users can review suggestions and corrections to improve their document quality.
- iii. **Multi-Format Support:** The system supports multiple document types, including project reports, theses, resumes, and cover letters. It ensures compatibility with both PDF and DOCX formats, allowing broad applicability in academic, professional, and administrative settings.

b) Non-Functional Requirements

- i. **Performance Requirement:** The system should process documents and generate validation reports within 5–10 seconds for standard-length documents (e.g., 10–20 pages).
- ii. **Accuracy Requirement:** The system should correctly identify and validate at least 90% of structural elements and required sections across all supported document types.
- iii. **Usability Requirement:** The interface should be web-based, intuitive, and user-friendly, enabling users with minimal technical knowledge to upload documents and view results easily.

- iv. **Security and Privacy Requirement:** All user documents and data must be processed locally or securely on the server, ensuring no sensitive content is leaked. The system should comply with standard data privacy and security guidelines.
- v. **Scalability and Extensibility:** The system architecture should allow future enhancements, such as grammar checking, AI-based content evaluation, or integration with learning management systems, without major redesign.

5.2 Feasibility Study

5.2.1 Technical Feasibility: The Document Requirement Validation System can be implemented using open-source Python libraries such as python-docx and pdfplumber for text extraction. Rule-based algorithms handle structure, headings, spacing, and formatting, while optional NLP libraries like spaCy can be added for grammar or content analysis. The system runs on standard computing resources without specialized hardware, and the web interface can be built with lightweight frameworks.

5.2.2 Operational Feasibility: The system operates as a backend service with a web interface, allowing users to upload documents and receive instant validation results. Actionable feedback and basic auto-corrections reduce manual effort. It is scalable for multiple document types, including reports, theses, resumes, and cover letters, and remains user-friendly for academic and professional users.

5.2.3 Economic Feasibility: Costs are limited to development time and server hosting. By eliminating reliance on manual validation and separate tools, the system provides long-term operational savings and improved efficiency. Open-source tools minimize software costs, making the project cost-effective, sustainable, and financially viable for institutions and organizations.

5.2.4 Schedule Feasibility: Gantt Chart

Tasks		Project Schedule						
	Months	February	March				April	
	Weeks	4	1	2	3	4	1	2
Requirement Analysis								
System Design and Architecture								
Backend Development								
Frontend Development								
Integration and Testing								
Documentation and Report Writing								

Figure 1: Gantt Chart

5.3 High-Level Design of System

5.3.1 Methodology of the Proposed System

The system will follow the Waterfall model, where development progresses step by step through clearly defined phases. This approach is ideal since the project requirements are well-established and straightforward. The workflow includes:

1. Requirement Analysis: Finalize functional and non-functional requirements.
2. System Design: Define architecture and module interactions.
3. Implementation: Develop backend, validation engine, and web interface.
4. Integration & Testing: Combine modules and test on multiple document types.
5. Deployment & Maintenance: Launch the system and plan for future updates.

5.3.2 Workflow of the Proposed System:

The proposed system follows a rule-based validation approach to assess user-submitted documents. It is designed to handle both academic and professional documents, including reports, theses, resumes, and cover letters, in PDF or DOCX formats. The workflow is as follows:

1. **Document Upload:** Users can upload a document for validation and optionally, a requirement specification document for customized evaluation.
2. **Text Extraction:** The system extracts text from the uploaded documents using Python libraries such as python-docx for DOCX files and pdfplumber for PDFs.

3. **Requirement Mapping / Rule Application:** Key sections and mandatory rules are identified automatically from the documents, or predefined rules are applied if no requirement document is provided.
4. **Validation & Scoring:** Each section of the document is checked for headings, structure, spacing, and formatting compliance. A section-wise score is calculated, and an overall validation score is generated.
5. **Feedback & Auto-Corrections:** The system generates actionable suggestions for sections that do not meet standards. Basic auto-corrections, such as adding missing headings, fixing spacing, or aligning formatting, are applied automatically.
6. **Output Generation:** Users receive a detailed report containing section-wise scores, remarks, suggestions for improvement, and corrected formatting where applicable.

5.3.3 System Flowchart

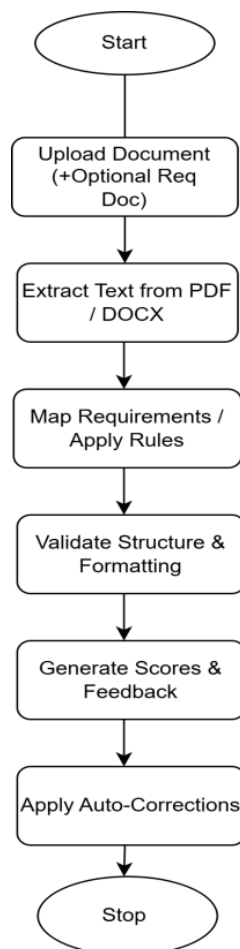


Figure 2: System Flowchart

5.3.4 System Architecture

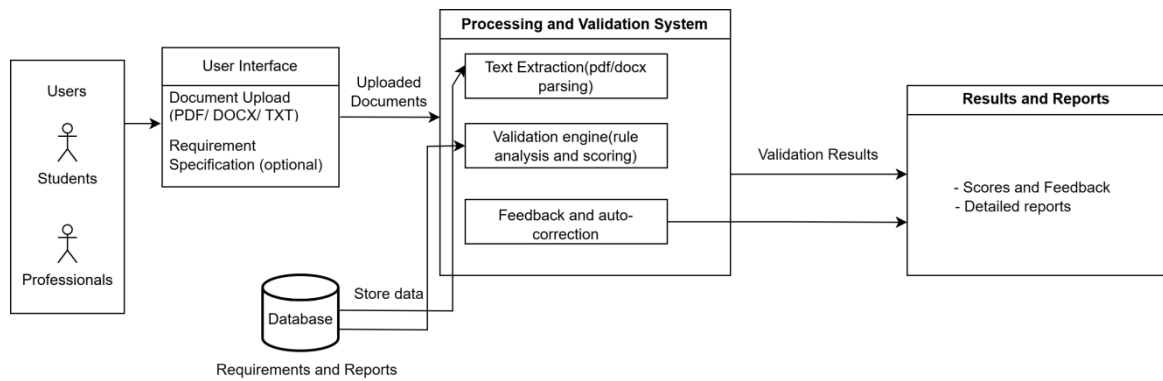


Figure 3: system architecture 1

5.3.5 Description of Algorithms

1. Requirement Extraction Algorithm

- Input: Optional requirement document.
- Process: Scan for mandatory sections (e.g., Abstract, Introduction, Methodology, Conclusion, etc) and extract rules for headings, spacing, formatting, and content.
- Output: A set of rules to validate the user's document.

2. Validation Algorithm

- Process: For each section in the document, check:
 - Required headings and sections
 - Proper spacing and formatting
 - Completeness according to extracted or predefined rules
- Output: Section-wise scores and overall validation score.

3. Feedback & Auto-Correction Algorithm

- Process: Identify missing sections or formatting errors. Apply basic corrections automatically, such as inserting missing headings and adjusting spacing and alignment
- Output: Actionable suggestions for improvement.

4. Scoring Mechanism

- Section Score: $(\text{Number of rules satisfied} / \text{Total applicable rules}) \times 100$
- Overall Score: Average of all section scores

6. Expected Outcomes

Upon successful implementation, the proposed Document Requirement Validation System is expected to achieve the following:

- **Comprehensive Document Validation:** Accurately evaluates documents such as reports, theses, resumes, and cover letters for structural completeness, headings, spacing, and formatting compliance. Validation is performed against either a user-uploaded requirement specification or predefined rules.
- **Section-Wise Scoring and Overall Evaluation:** Provides individual section scores and an overall validation score, allowing users to quickly assess document quality and identify strengths and weaknesses.
- **Actionable Feedback and Suggestions:** Generates detailed, section-wise feedback with recommendations for restructuring, formatting, or completing missing sections.
- **Basic Auto-Corrections:** Automatically corrects minor errors, such as missing headings, spacing issues, and alignment, reducing the need for manual editing.
- **Enhanced Efficiency and Consistency:** Reduces time and effort compared to manual validation and ensures consistent evaluation across multiple documents.
- **Scalability and Future Enhancements:** System architecture allows integration of advanced features, such as grammar and style checks, AI-based evaluation, and summarization, adapting to evolving academic and professional requirements.
- **Improved Learning and Document Quality:** Provides actionable insights to help users produce well-structured, compliant, and professional-quality documents.

7. References

- [1] M. Abisornam, "Document Verification System," International Journal of Scientific Research in Engineering and Management, vol. 9, pp. 1–9, 2025, doi:10.55041/IJSREM46911.
- [2] "DocVerify: Automated Documents Verification System," ISJEM Journal. [Online]. Available: <https://isjem.com/download/docverify-automated-documents-verification-system/>
- [3] ElectroNeek, "Human Validation of Document Processing," Oct. 18, 2024. [Online]. Available: <https://electroneek.com/use-case/human-validation-of-document-processing/>
- [4] D. Jurafsky and J. H. Martin, Speech and Language Processing, 3rd ed. draft, 2023.
- [5] Y. Xu et al., "LayoutLM: Pre-training of Text and Layout for Document Image Understanding," in Proc. KDD, 2020.
- [6] A. Katti et al., "Chargrid: Towards Understanding 2D Documents," in Proc. EMNLP, 2018.
- [7] R. Smith, "An Overview of the Tesseract OCR Engine," in Proc. ICDAR, 2007.